

# Module 7: Plotting and Visualization (Matplotlib and Seaborn)

**Author:** Lei Wu

**Date:** 2023-01-01

**Version:** 1.0.0

## Learning Objectives

By the end of this module, you will be able to:

- **Master Matplotlib fundamentals:** Create, customize, and style various plot types
- **Apply statistical visualization:** Use Seaborn for advanced statistical graphics
- **Design publication-quality plots:** Implement professional styling and layout techniques
- **Conduct exploratory data analysis:** Use visualization to discover patterns and insights
- **Export and share visualizations:** Save plots in multiple formats for different purposes

## Module Overview

This module provides comprehensive coverage of data visualization in Python, progressing from basic plotting concepts to advanced statistical graphics. You'll learn to create publication-quality visualizations that effectively communicate data insights.

## Topics Covered

1. **Introduction to Matplotlib** - Foundation and core concepts
2. **Basic Plotting Functions** - Essential plot types and syntax
3. **Figure and Axes Management** - Object-oriented plotting approach
4. **Plot Customization and Styling** - Professional appearance and branding
5. **Different Plot Types** - Line, bar, scatter, histogram, and more
6. **Introduction to Seaborn** - Statistical visualization library
7. **Statistical Plots and Distributions** - Advanced statistical graphics
8. **Categorical Plots** - Visualizing categorical data effectively
9. **Matrix Plots and Heatmaps** - Correlation and pattern visualization
10. **Subplots and Multiple Plots** - Complex layouts and multi-panel figures
11. **Saving and Exporting Plots** - Production-ready output formats

## Prerequisites

- Basic Python programming (Modules 1-2)
- NumPy fundamentals (Module 3)
- Pandas basics (Modules 4-6)

## Key Libraries

- **Matplotlib:** Core plotting library with fine-grained control
- **Seaborn:** High-level statistical visualization
- **NumPy:** Numerical data generation and manipulation
- **Pandas:** Data handling and integration

## 1. Introduction to Matplotlib

### What is Matplotlib?

Matplotlib is the foundational plotting library in Python, providing a MATLAB-like interface for creating static, animated, and interactive visualizations. It's the backbone of Python's data visualization ecosystem.

### Key Features

- **Multi-platform:** Works on Windows, macOS, and Linux
- **Multiple backends:** Supports various output formats (PNG, PDF, SVG, etc.)
- **Object-oriented and procedural interfaces:** Flexible programming approaches
- **Integration:** Works seamlessly with NumPy, Pandas, and Jupyter notebooks
- **Customization:** Fine-grained control over every aspect of plots

### Matplotlib Architecture

Matplotlib follows a hierarchical structure:

- **Figure:** The top-level container for all plot elements
- **Axes:** The actual plotting area where data is visualized
- **Axis:** The numerical axis (x-axis, y-axis)
- **Artist:** Individual elements like lines, text, patches

In [ ...

```
1      # Import necessary libraries
2      import matplotlib.pyplot as plt
3      import numpy as np
4      import pandas as pd
5      import seaborn as sns
6      from matplotlib.patches import Rectangle, Circle
7      import warnings
8      warnings.filterwarnings('ignore')
9
10
11     # Set up the plotting style
12     plt.style.use('default')
13     sns.set_palette("husl")
```

```

14
15 # 1. Introduction to Matplotlib - Basic Setup
16 print("=== INTRODUCTION TO MATPLOTLIB ===")
17
18 # Basic import and setup
19 print("Matplotlib version:", plt.matplotlib.__version__)
20
21 # Create a simple plot to demonstrate basic functionality
22 x = np.linspace(0, 10, 100)
23 y = np.sin(x)
24
25 # Method 1: Pyplot interface (procedural)
26 plt.figure(figsize=(8, 6))
27 plt.plot(x, y, label='sin(x)', linewidth=2)
28 plt.title('Basic Matplotlib Plot')
29 plt.xlabel('x')
30 plt.ylabel('y')
31 plt.legend()
32 plt.grid(True, alpha=0.3)
33 plt.show()
34
35 # Method 2: Object-oriented interface
36 fig, ax = plt.subplots(figsize=(8, 6))
37 ax.plot(x, y, label='sin(x)', linewidth=2)
38 ax.set_title('Object-Oriented Matplotlib Plot')
39 ax.set_xlabel('x')
40 ax.set_ylabel('y')
41 ax.legend()
42 ax.grid(True, alpha=0.3)
43 plt.show()
44
45 # Understanding the matplotlib hierarchy
46 print("\nMatplotlib Hierarchy:")
47 print("Figure -> Axes -> Axis -> Artist")
48 print("- Figure: Top-level container")
49 print("- Axes: Plotting area")
50 print("- Axis: X/Y axis")
51 print("- Artist: Individual elements")
52
53 # Demonstrate figure and axes properties
54 fig, ax = plt.subplots(figsize=(10, 6))
55
56 # Add some data
57 x = np.linspace(0, 2*np.pi, 100)
58 y1 = np.sin(x)
59 y2 = np.cos(x)
60
61 # Plot multiple lines
62 ax.plot(x, y1, label='sin(x)', linewidth=2, color='blue')
63 ax.plot(x, y2, label='cos(x)', linewidth=2, color='red')
64
65 # Customize the plot
66 ax.set_title('Understanding Matplotlib Components', fontsize=16,
67 ax.set_xlabel('Angle (radians)', fontsize=12)
68 ax.set_ylabel('Value', fontsize=12)
69 ax.legend(fontsize=12)
70 ax.grid(True, alpha=0.3)
71
72 # Add annotations
73 ax.annotate('Maximum', xy=(np.pi/2, 1), xytext=(np.pi/2 + 0.5, 1.

```

```
78         arrowprops=dict(arrowstyle='->', color='black'),
79         fontsize=10, ha='center')
80
plt.tight_layout()
plt.show()

print("\n=== END OF MATPLOTLIB INTRODUCTION ===")
```

## 2. Basic Plotting Functions

### Essential Plot Types

Matplotlib provides a comprehensive set of plotting functions for different data types and visualization needs:

#### Line Plots (`plt.plot()`)

- **Purpose:** Show trends and relationships over continuous data
- **Use cases:** Time series, mathematical functions, continuous relationships
- **Key parameters:** `x`, `y`, `color`, `linewidth`, `linestyle`, `marker`

#### Scatter Plots (`plt.scatter()`)

- **Purpose:** Show relationships between two variables
- **Use cases:** Correlation analysis, clustering, outlier detection
- **Key parameters:** `x`, `y`, `c` (color), `s` (size), `alpha` (transparency)

#### Bar Plots (`plt.bar()`)

- **Purpose:** Compare categorical data
- **Use cases:** Counts, frequencies, comparisons
- **Key parameters:** `x`, `height`, `width`, `color`, `edgecolor`

#### Histograms (`plt.hist()`)

- **Purpose:** Show distribution of numerical data
- **Use cases:** Data exploration, distribution analysis
- **Key parameters:** `x`, `bins`, `alpha`, `density`, `cumulative`

#### Pie Charts (`plt.pie()`)

- **Purpose:** Show proportions of a whole
- **Use cases:** Composition analysis, market share
- **Key parameters:** `x`, `labels`, `autopct`, `explode`, `colors`

```

1  # 2. Basic Plotting Functions - Comprehensive Examples
2  print("=== BASIC PLOTTING FUNCTIONS ===")
3
4
5  # Generate sample data
6  np.random.seed(42)
7  x = np.linspace(0, 10, 50)
8  y1 = np.sin(x)
9  y2 = np.cos(x)
10 y3 = np.sin(x) * np.cos(x)
11
12 # Create a comprehensive example showing all basic plot types
13 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
14 fig.suptitle('Basic Matplotlib Plot Types', fontsize=16, fontweig
15
16 # 1. Line Plot
17 axes[0, 0].plot(x, y1, 'b--', linewidth=2, label='sin(x)')
18 axes[0, 0].plot(x, y2, 'r--', linewidth=2, label='cos(x)')
19 axes[0, 0].set_title('Line Plot')
20 axes[0, 0].set_xlabel('x')
21 axes[0, 0].set_ylabel('y')
22 axes[0, 0].legend()
23 axes[0, 0].grid(True, alpha=0.3)
24
25 # 2. Scatter Plot
26 x_scatter = np.random.randn(100)
27 y_scatter = 2 * x_scatter + np.random.randn(100) * 0.5
28 colors = np.random.rand(100)
29 axes[0, 1].scatter(x_scatter, y_scatter, c=colors, alpha=0.6, s=5
30 axes[0, 1].set_title('Scatter Plot')
31 axes[0, 1].set_xlabel('X')
32 axes[0, 1].set_ylabel('Y')
33
34 # 3. Bar Plot
35 categories = ['A', 'B', 'C', 'D', 'E']
36 values = [23, 45, 56, 78, 32]
37 colors_bar = ['red', 'green', 'blue', 'orange', 'purple']
38 axes[0, 2].bar(categories, values, color=colors_bar, edgecolor='b
39 axes[0, 2].set_title('Bar Plot')
40 axes[0, 2].set_xlabel('Categories')
41 axes[0, 2].set_ylabel('Values')
42
43 # 4. Histogram
44 data_hist = np.random.normal(0, 1, 1000)
45 axes[1, 0].hist(data_hist, bins=30, alpha=0.7, color='skyblue', e
46 axes[1, 0].set_title('Histogram')
47 axes[1, 0].set_xlabel('Value')
48 axes[1, 0].set_ylabel('Frequency')
49
50 # 5. Pie Chart
51 sizes = [30, 25, 20, 15, 10]
52 labels = ['Category 1', 'Category 2', 'Category 3', 'Category 4',
53 colors_pie = ['gold', 'lightcoral', 'lightskyblue', 'lightgreen',
54 explode = (0.1, 0, 0, 0, 0) # explode the first slice
55 axes[1, 1].pie(sizes, labels=labels, colors=colors_pie, autopct='
56 explode=explode, shadow=True, startangle=90)
57 axes[1, 1].set_title('Pie Chart')
58
59 # 6. Box Plot

```

```

63 data_box = [np.random.normal(0, 1, 100),
64             np.random.normal(1, 1.5, 100),
65             np.random.normal(2, 0.5, 100)]
66 axes[1, 2].boxplot(data_box, labels=['Group 1', 'Group 2', 'Group
67 axes[1, 2].set_title('Box Plot')
68 axes[1, 2].set_ylabel('Value')
69
70 plt.tight_layout()
71 plt.show()
72
73 # Advanced Line Plot Examples
74 print("\nAdvanced Line Plot Examples:")
75
76 # Create a more sophisticated line plot
77 fig, ax = plt.subplots(figsize=(12, 8))
78
79 # Multiple lines with different styles
80 x = np.linspace(0, 4*np.pi, 200)
81 y1 = np.sin(x)
82 y2 = np.cos(x)
83 y3 = np.sin(x + np.pi/4)
84 y4 = np.cos(x + np.pi/4)
85
86 # Plot with different line styles and markers
87 ax.plot(x, y1, 'b-', linewidth=2, label='sin(x)', alpha=0.8)
88 ax.plot(x, y2, 'r--', linewidth=2, label='cos(x)', alpha=0.8)
89 ax.plot(x, y3, 'g:', linewidth=3, label='sin(x +  $\pi/4$ )', alpha=0.8)
90 ax.plot(x, y4, 'm-.', linewidth=2, label='cos(x +  $\pi/4$ )', alpha=0.8)
91
92 # Add markers at specific points
93 marker_x = np.linspace(0, 4*np.pi, 20)
94 ax.plot(marker_x, np.sin(marker_x), 'ko', markersize=6, alpha=0.6)
95
96 # Customize the plot
97 ax.set_title('Advanced Line Plot with Multiple Styles', fontsize=
98 ax.set_xlabel('Angle (radians)', fontsize=12)
99 ax.set_ylabel('Value', fontsize=12)
100 ax.legend(fontsize=10, loc='upper right')
101 ax.grid(True, alpha=0.3)
102 ax.set_xlim(0, 4*np.pi)
103
104 # Add horizontal and vertical lines
105 ax.axhline(y=0, color='black', linestyle='-', alpha=0.3)
106 ax.axvline(x=2*np.pi, color='red', linestyle='--', alpha=0.5, lab
107
108 plt.tight_layout()
109 plt.show()
110
111 # Scatter Plot with Color Mapping
112 print("\nAdvanced Scatter Plot with Color Mapping:")
113
114 # Generate data for scatter plot
115 n = 100
116 x_scatter = np.random.randn(n)
117 y_scatter = 2 * x_scatter + np.random.randn(n) * 0.5
118 colors_scatter = np.random.rand(n)
119 sizes_scatter = 1000 * np.random.rand(n)
120
121 fig, ax = plt.subplots(figsize=(10, 6))
122 scatter = ax.scatter(x_scatter, y_scatter, c=colors_scatter, s=sizes_scatter)

```

```

126         alpha=0.6, cmap='viridis')
127     ax.set_title('Advanced Scatter Plot with Color and Size Mapping')
    ax.set_xlabel('X')
    ax.set_ylabel('Y')
    plt.colorbar(scatter, label='Color Value')
    plt.show()

    print("\n=== END OF BASIC PLOTTING FUNCTIONS ===")

```

## 3. Figure and Axes Management

### Object-Oriented Approach

The object-oriented approach provides more control and is recommended for complex plots:

#### Figure Management

- **plt.figure()** : Create a new figure
- **fig.add\_subplot()** : Add subplots to a figure
- **plt.subplots()** : Create figure and axes simultaneously
- **fig.savefig()** : Save figure to file

#### Axes Management

- **ax.plot()** : Plot on specific axes
- **ax.set\_xlabel()** : Set x-axis label
- **ax.set\_ylabel()** : Set y-axis label
- **ax.set\_title()** : Set plot title
- **ax.legend()** : Add legend
- **ax.grid()** : Add grid

#### Subplot Layouts

- **plt.subplot(nrows, ncols, index)** : Create subplots
- **plt.subplots(nrows, ncols)** : Create multiple subplots
- **plt.subplot2grid()** : Create subplots with custom grid
- **plt.GridSpec()** : Advanced subplot layouts

In [ ...

```

1      # 3. Figure and Axes Management – Object-Oriented Approach
2      print("=== FIGURE AND AXES MANAGEMENT ===")
3
4
5      # Method 1: Using plt.subplots() – Most Common
6      print("Method 1: Using plt.subplots()")
7      fig, axes = plt.subplots(2, 2, figsize=(12, 10))
8      fig.suptitle('Figure and Axes Management – Method 1', fontsize=16)
9
10     # Generate data
11

```

```

12 x = np.linspace(0, 2*np.pi, 100)
13 y1 = np.sin(x)
14 y2 = np.cos(x)
15 y3 = np.tan(x)
16 y4 = np.exp(-x/5) * np.sin(x)
17
18 # Plot on different axes
19 axes[0, 0].plot(x, y1, 'b-', linewidth=2)
20 axes[0, 0].set_title('sin(x)')
21 axes[0, 0].set_xlabel('x')
22 axes[0, 0].set_ylabel('y')
23 axes[0, 0].grid(True, alpha=0.3)
24
25 axes[0, 1].plot(x, y2, 'r-', linewidth=2)
26 axes[0, 1].set_title('cos(x)')
27 axes[0, 1].set_xlabel('x')
28 axes[0, 1].set_ylabel('y')
29 axes[0, 1].grid(True, alpha=0.3)
30
31 axes[1, 0].plot(x, y3, 'g-', linewidth=2)
32 axes[1, 0].set_title('tan(x)')
33 axes[1, 0].set_xlabel('x')
34 axes[1, 0].set_ylabel('y')
35 axes[1, 0].grid(True, alpha=0.3)
36 axes[1, 0].set_ylim(-5, 5) # Limit y-axis for tan function
37
38 axes[1, 1].plot(x, y4, 'm-', linewidth=2)
39 axes[1, 1].set_title('exp(-x/5) * sin(x)')
40 axes[1, 1].set_xlabel('x')
41 axes[1, 1].set_ylabel('y')
42 axes[1, 1].grid(True, alpha=0.3)
43
44 plt.tight_layout()
45 plt.show()
46
47 # Method 2: Using plt.subplot() - Individual subplots
48 print("\nMethod 2: Using plt.subplot()")
49 plt.figure(figsize=(12, 8))
50
51 # First subplot
52 plt.subplot(2, 3, 1)
53 plt.plot(x, y1, 'b-', linewidth=2)
54 plt.title('sin(x)')
55 plt.grid(True, alpha=0.3)
56
57 # Second subplot
58 plt.subplot(2, 3, 2)
59 plt.plot(x, y2, 'r-', linewidth=2)
60 plt.title('cos(x)')
61 plt.grid(True, alpha=0.3)
62
63 # Third subplot
64 plt.subplot(2, 3, 3)
65 plt.plot(x, y3, 'g-', linewidth=2)
66 plt.title('tan(x)')
67 plt.grid(True, alpha=0.3)
68 plt.ylim(-5, 5)
69
70 # Fourth subplot (spans 2 columns)
71 plt.subplot(2, 3, (4, 5))

```

```

76 plt.plot(x, y4, 'm-', linewidth=2)
77 plt.title('exp(-x/5) * sin(x)')
78 plt.grid(True, alpha=0.3)
79
80 # Sixth subplot
81 plt.subplot(2, 3, 6)
82 plt.hist(np.random.normal(0, 1, 1000), bins=30, alpha=0.7, color=
83 plt.title('Normal Distribution')
84 plt.grid(True, alpha=0.3)
85
86 plt.suptitle('Figure and Axes Management - Method 2', fontsize=16
87 plt.tight_layout()
88 plt.show()
89
90 # Method 3: Using subplot2grid for complex layouts
91 print("\nMethod 3: Using subplot2grid for complex layouts")
92 fig = plt.figure(figsize=(15, 10))
93
94 # Main plot (large)
95 ax1 = plt.subplot2grid((3, 3), (0, 0), colspan=2, rowspan=2)
96 ax1.plot(x, y1, 'b-', linewidth=2, label='sin(x)')
97 ax1.plot(x, y2, 'r-', linewidth=2, label='cos(x)')
98 ax1.set_title('Main Plot - Trigonometric Functions', fontsize=14,
99 ax1.set_xlabel('x')
100 ax1.set_ylabel('y')
101 ax1.legend()
102 ax1.grid(True, alpha=0.3)
103
104 # Small plots
105 ax2 = plt.subplot2grid((3, 3), (0, 2))
106 ax2.hist(np.random.normal(0, 1, 500), bins=20, alpha=0.7, color='
107 ax2.set_title('Distribution')
108
109 ax3 = plt.subplot2grid((3, 3), (1, 2))
110 ax3.bar(['A', 'B', 'C', 'D'], [23, 45, 56, 78], color=['red', 'gr
111 ax3.set_title('Bar Chart')
112
113 # Bottom plot (spans full width)
114 ax4 = plt.subplot2grid((3, 3), (2, 0), colspan=3)
115 ax4.plot(x, y4, 'm-', linewidth=2)
116 ax4.fill_between(x, y4, alpha=0.3, color='magenta')
117 ax4.set_title('Damped Oscillation', fontsize=12)
118 ax4.set_xlabel('x')
119 ax4.set_ylabel('y')
120 ax4.grid(True, alpha=0.3)
121
122 plt.suptitle('Complex Layout with subplot2grid', fontsize=16, fon
123 plt.tight_layout()
124 plt.show()
125
126 # Method 4: Using GridSpec for advanced layouts
127 print("\nMethod 4: Using GridSpec for advanced layouts")
128 from matplotlib.gridspec import GridSpec
129
130 fig = plt.figure(figsize=(15, 10))
131 gs = GridSpec(3, 3, figure=fig)
132
133 # Create axes with GridSpec
134 ax1 = fig.add_subplot(gs[0, :]) # Top row, all columns
135 ax2 = fig.add_subplot(gs[1, 0]) # Middle row, first column

```

```

140 ax3 = fig.add_subplot(gs[1, 1]) # Middle row, second column
141 ax4 = fig.add_subplot(gs[1, 2]) # Middle row, third column
142 ax5 = fig.add_subplot(gs[2, :]) # Bottom row, all columns
143
144 # Plot on each axis
145 ax1.plot(x, y1, 'b-', linewidth=2, label='sin(x)')
146 ax1.plot(x, y2, 'r-', linewidth=2, label='cos(x)')
147 ax1.set_title('Top Plot - Trigonometric Functions')
148 ax1.legend()
149 ax1.grid(True, alpha=0.3)
150
151 ax2.hist(np.random.normal(0, 1, 500), bins=20, alpha=0.7, color='green')
152 ax2.set_title('Histogram')
153
154 ax3.bar(['A', 'B', 'C'], [23, 45, 56], color=['red', 'green', 'blue'])
155 ax3.set_title('Bar Chart')
156
157 ax4.pie([30, 25, 20, 15, 10], labels=['A', 'B', 'C', 'D', 'E'], autopct='%1.1f%%')
158 ax4.set_title('Pie Chart')
159
160 ax5.scatter(np.random.randn(100), np.random.randn(100), alpha=0.6)
161 ax5.set_title('Scatter Plot')
162 ax5.grid(True, alpha=0.3)
163
164 plt.suptitle('Advanced Layout with GridSpec', fontsize=16, fontweight='bold')
165 plt.tight_layout()
166 plt.show()
167
168 # Figure properties and customization
169 print("\nFigure Properties and Customization:")
170 fig, ax = plt.subplots(figsize=(10, 6))
171
172 # Plot data
173 x = np.linspace(0, 10, 100)
174 y = np.sin(x) * np.exp(-x/10)
175
176 ax.plot(x, y, 'b-', linewidth=2, label='Damped Sine Wave')
177 ax.fill_between(x, y, alpha=0.3, color='blue')
178
179 # Customize the plot
180 ax.set_title('Figure Properties and Customization', fontsize=16, fontweight='bold')
181 ax.set_xlabel('Time', fontsize=12)
182 ax.set_ylabel('Amplitude', fontsize=12)
183 ax.legend(fontsize=12)
184 ax.grid(True, alpha=0.3)
185
186 # Set axis limits and ticks
187 ax.set_xlim(0, 10)
188 ax.set_ylim(-1, 1)
189 ax.set_xticks(np.arange(0, 11, 2))
190 ax.set_yticks(np.arange(-1, 1.1, 0.5))
191
192 # Add text annotation
193 ax.text(5, 0.5, 'Peak Value', fontsize=12, ha='center',
194        bbox=dict(boxstyle='round', facecolor='yellow', alpha=0.7))
195
196 # Add arrow annotation
197 ax.annotate('Decay', xy=(8, 0.2), xytext=(6, 0.6),
198            arrowprops=dict(arrowstyle='->', color='red', lw=2),
199            fontsize=12, ha='center')

```

```
plt.tight_layout()
plt.show()

print("\n=== END OF FIGURE AND AXES MANAGEMENT ===")
```

## 4. Plot Customization and Styling

### Professional Appearance

Creating publication-quality plots requires attention to styling details:

#### Color Management

- **Named colors:** 'red', 'blue', 'green', etc.
- **Hex colors:** '#FF5733', '#2E86AB'
- **RGB tuples:** (0.1, 0.2, 0.5)
- **Colormaps:** 'viridis', 'plasma', 'inferno', 'coolwarm'

#### Typography

- **Font families:** 'serif', 'sans-serif', 'monospace'
- **Font sizes:** title, labels, ticks, legend
- **Font weights:** 'normal', 'bold', 'light'
- **Text alignment:** 'left', 'center', 'right'

#### Line and Marker Styles

- **Line styles:** '-', '--', '-.', ':', 'None'
- **Markers:** 'o', 's', '^', 'v', 'D', '\*', '+'
- **Line widths:** 0.5, 1, 2, 3, etc.
- **Marker sizes:** 1, 5, 10, 20, etc.

#### Grid and Background

- **Grid styles:** Major/minor grids, transparency
- **Background colors:** Face, edge colors
- **Spine customization:** Top, bottom, left, right

In [ ...

```
1  # 4. Plot Customization and Styling – Professional Appearance
2  print("=== PLOT CUSTOMIZATION AND STYLING ===")
3
4
5  # Color Management Examples
6  print("Color Management Examples:")
7  fig, axes = plt.subplots(2, 2, figsize=(15, 10))
8  fig.suptitle('Color Management and Styling', fontsize=16, fontwei
9
10 # Generate data
```

```

11 x = np.linspace(0, 2*np.pi, 100)
12 y1 = np.sin(x)
13 y2 = np.cos(x)
14 y3 = np.sin(x + np.pi/4)
15 y4 = np.cos(x + np.pi/4)
16
17 # 1. Named Colors
18 axes[0, 0].plot(x, y1, color='red', linewidth=2, label='sin(x)')
19 axes[0, 0].plot(x, y2, color='blue', linewidth=2, label='cos(x)')
20 axes[0, 0].set_title('Named Colors')
21 axes[0, 0].legend()
22 axes[0, 0].grid(True, alpha=0.3)
23
24 # 2. Hex Colors
25 axes[0, 1].plot(x, y1, color='#FF5733', linewidth=2, label='sin(x)')
26 axes[0, 1].plot(x, y2, color='#2E86AB', linewidth=2, label='cos(x)')
27 axes[0, 1].set_title('Hex Colors')
28 axes[0, 1].legend()
29 axes[0, 1].grid(True, alpha=0.3)
30
31 # 3. RGB Tuples
32 axes[1, 0].plot(x, y1, color=(0.1, 0.2, 0.5), linewidth=2, label='sin(x)')
33 axes[1, 0].plot(x, y2, color=(0.5, 0.1, 0.2), linewidth=2, label='cos(x)')
34 axes[1, 0].set_title('RGB Tuples')
35 axes[1, 0].legend()
36 axes[1, 0].grid(True, alpha=0.3)
37
38 # 4. Colormap
39 colors = plt.cm.viridis(np.linspace(0, 1, 4))
40 axes[1, 1].plot(x, y1, color=colors[0], linewidth=2, label='sin(x)')
41 axes[1, 1].plot(x, y2, color=colors[1], linewidth=2, label='cos(x)')
42 axes[1, 1].plot(x, y3, color=colors[2], linewidth=2, label='sin(x)')
43 axes[1, 1].plot(x, y4, color=colors[3], linewidth=2, label='cos(x)')
44 axes[1, 1].set_title('Colormap (viridis)')
45 axes[1, 1].legend()
46 axes[1, 1].grid(True, alpha=0.3)
47
48 plt.tight_layout()
49 plt.show()
50
51 # Typography and Font Management
52 print("\nTypography and Font Management:")
53 fig, ax = plt.subplots(figsize=(12, 8))
54
55 # Generate data
56 x = np.linspace(0, 10, 100)
57 y1 = np.sin(x)
58 y2 = np.cos(x)
59
60 # Plot with custom typography
61 ax.plot(x, y1, 'b-', linewidth=3, label='Sine Wave', alpha=0.8)
62 ax.plot(x, y2, 'r--', linewidth=3, label='Cosine Wave', alpha=0.8)
63
64 # Customize typography
65 ax.set_title('Typography and Font Management',
66             fontsize=20, fontweight='bold', fontfamily='serif',
67             fontstyle='italic')
68 ax.set_xlabel('Time (seconds)', fontsize=14, fontweight='bold', fontfamily='serif',
69             fontstyle='italic')
70 ax.set_ylabel('Amplitude', fontsize=14, fontweight='bold', fontfamily='serif',
71             fontstyle='italic')
72
73 # Customize legend
74

```

```

75 ax.legend(fontsize=12, fontfamily='monospace', frameon=True,
76           fancybox=True, shadow=True, loc='upper right')
77
78 # Customize tick labels
79 ax.tick_params(axis='both', which='major', labelsize=12, fontfamily='monospace')
80 ax.set_xticks(np.arange(0, 11, 2))
81 ax.set_yticks(np.arange(-1, 1.1, 0.5))
82
83 # Add text annotations with different fonts
84 ax.text(2, 0.8, 'Peak Value', fontsize=14, fontweight='bold',
85        fontfamily='serif', ha='center',
86        bbox=dict(boxstyle='round,pad=0.5', facecolor='yellow', a
87
88
89 ax.text(8, -0.8, 'Trough Value', fontsize=14, fontweight='bold',
90        fontfamily='serif', ha='center',
91        bbox=dict(boxstyle='round,pad=0.5', facecolor='lightblue'
92
93 ax.grid(True, alpha=0.3, linestyle='--', linewidth=0.5)
94 plt.tight_layout()
95 plt.show()
96
97 # Line and Marker Styles
98 print("\nLine and Marker Styles:")
99 fig, axes = plt.subplots(2, 2, figsize=(15, 10))
100 fig.suptitle('Line and Marker Styles', fontsize=16, fontweight='b
101
102 # Generate data
103 x = np.linspace(0, 10, 20)
104 y = np.sin(x)
105
106 # 1. Line Styles
107 line_styles = ['-', '--', '-.', ':']
108 colors = ['blue', 'red', 'green', 'purple']
109 for i, (style, color) in enumerate(zip(line_styles, colors)):
110     axes[0, 0].plot(x, y + i*0.5, linestyle=style, color=color,
111                    linewidth=2, label=f'Style: {style}')
112 axes[0, 0].set_title('Line Styles')
113 axes[0, 0].legend()
114 axes[0, 0].grid(True, alpha=0.3)
115
116 # 2. Marker Styles
117 markers = ['o', 's', '^', 'v', 'D', '*', '+', 'x']
118 colors = plt.cm.tab10(np.linspace(0, 1, len(markers)))
119 for i, (marker, color) in enumerate(zip(markers, colors)):
120     axes[0, 1].plot(x, y + i*0.3, marker=marker, color=color,
121                    markersize=8, linestyle='None', label=f'Marke
122 axes[0, 1].set_title('Marker Styles')
123 axes[0, 1].legend()
124 axes[0, 1].grid(True, alpha=0.3)
125
126 # 3. Line Widths
127 widths = [0.5, 1, 2, 3, 4]
128 colors = plt.cm.viridis(np.linspace(0, 1, len(widths)))
129 for i, (width, color) in enumerate(zip(widths, colors)):
130     axes[1, 0].plot(x, y + i*0.3, linewidth=width, color=color,
131                    label=f'Width: {width}')
132 axes[1, 0].set_title('Line Widths')
133 axes[1, 0].legend()
134 axes[1, 0].grid(True, alpha=0.3)
135
136
137
138

```

```

139 # 4. Marker Sizes
140 sizes = [5, 10, 15, 20, 25]
141 colors = plt.cm.plasma(np.linspace(0, 1, len(sizes)))
142 for i, (size, color) in enumerate(zip(sizes, colors)):
143     axes[1, 1].plot(x, y + i*0.3, marker='o', markersize=size, co
144                     linestyle='None', label=f'Size: {size}')
145 axes[1, 1].set_title('Marker Sizes')
146 axes[1, 1].legend()
147 axes[1, 1].grid(True, alpha=0.3)
148
149 plt.tight_layout()
150 plt.show()
151
152 # Advanced Styling – Professional Plot
153 print("\nAdvanced Styling – Professional Plot:")
154 fig, ax = plt.subplots(figsize=(12, 8))
155
156 # Generate data
157 x = np.linspace(0, 4*np.pi, 200)
158 y1 = np.sin(x) * np.exp(-x/10)
159 y2 = np.cos(x) * np.exp(-x/10)
160 y3 = np.sin(x + np.pi/4) * np.exp(-x/10)
161
162 # Plot with advanced styling
163 ax.plot(x, y1, color='#2E86AB', linewidth=3, label='Damped Sine',
164         ax.plot(x, y2, color='#A23B72', linewidth=3, label='Damped Cosine',
165         ax.plot(x, y3, color='#F18F01', linewidth=3, label='Phase Shifted
166
167 # Fill between curves
168 ax.fill_between(x, y1, alpha=0.2, color='#2E86AB')
169 ax.fill_between(x, y2, alpha=0.2, color='#A23B72')
170
171 # Customize the plot
172 ax.set_title('Professional Damped Oscillations',
173             fontsize=18, fontweight='bold', fontfamily='serif',
174             ax.set_xlabel('Time (s)', fontsize=14, fontweight='bold')
175             ax.set_ylabel('Amplitude', fontsize=14, fontweight='bold')
176
177 # Customize legend
178 ax.legend(fontsize=12, frameon=True, fancybox=True, shadow=True,
179          loc='upper right', framealpha=0.9)
180
181 # Customize grid
182 ax.grid(True, alpha=0.3, linestyle='--', linewidth=0.5, color='gray')
183 ax.set_axisbelow(True)
184
185 # Customize spines
186 for spine in ax.spines.values():
187     spine.set_linewidth(1.5)
188     spine.set_color('black')
189
190 # Set axis limits and ticks
191 ax.set_xlim(0, 4*np.pi)
192 ax.set_ylim(-1.2, 1.2)
193 ax.set_xticks(np.arange(0, 4*np.pi + 1, np.pi))
194 ax.set_xticklabels(['0', ' $\pi$ ', '2 $\pi$ ', '3 $\pi$ ', '4 $\pi$ '])
195 ax.set_yticks(np.arange(-1, 1.1, 0.5))
196
197 # Add annotations
198 ax.annotate('Peak', xy=(np.pi/2, 1), xytext=(np.pi/2 + 1, 1.1),

```

```

202         arrowprops=dict(arrowstyle='->', color='red', lw=2),
203         fontsize=12, ha='center', fontweight='bold')

    # Add text box
    ax.text(0.02, 0.98, 'Damped Oscillations\nwith Phase Shift',
            transform=ax.transAxes, fontsize=12, fontweight='bold',
            verticalalignment='top', bbox=dict(boxstyle='round,pad=0.5',
            facecolor='lightblue', alpha=0.8))

    plt.tight_layout()
    plt.show()

    print("\n=== END OF PLOT CUSTOMIZATION AND STYLING ===")

```

## 5. Different Plot Types

### Comprehensive Plot Gallery

Matplotlib offers a wide variety of plot types for different data visualization needs:

#### Statistical Plots

- **Histograms:** Distribution of numerical data
- **Box plots:** Statistical summaries and outliers
- **Violin plots:** Distribution shapes
- **Error bars:** Uncertainty visualization
- **Contour plots:** 3D data in 2D

#### Categorical Plots

- **Bar charts:** Categorical comparisons
- **Horizontal bar charts:** Long category names
- **Stacked bar charts:** Part-to-whole relationships
- **Grouped bar charts:** Multiple categories

#### Time Series Plots

- **Line plots:** Trends over time
- **Area plots:** Cumulative values
- **Stacked area plots:** Multiple time series
- **Candlestick plots:** Financial data

#### 2D and 3D Plots

- **Scatter plots:** Two-variable relationships
- **Bubble charts:** Three-variable relationships
- **Heatmaps:** Matrix data visualization
- **3D surface plots:** Three-dimensional data

In [ ...

```
1  # 5. Different Plot Types - Comprehensive Gallery
2  print("=== DIFFERENT PLOT TYPES ===")
3
4
5  # Generate comprehensive sample data
6  np.random.seed(42)
7  n_samples = 1000
8
9  # Statistical data
10 data_normal = np.random.normal(0, 1, n_samples)
11 data_exponential = np.random.exponential(2, n_samples)
12 data_uniform = np.random.uniform(-2, 2, n_samples)
13
14 # Categorical data
15 categories = ['A', 'B', 'C', 'D', 'E']
16 values1 = [23, 45, 56, 78, 32]
17 values2 = [15, 25, 35, 45, 55]
18 values3 = [10, 20, 30, 40, 50]
19
20 # Time series data
21 time = np.linspace(0, 10, 100)
22 ts1 = np.sin(time) + 0.1 * np.random.randn(100)
23 ts2 = np.cos(time) + 0.1 * np.random.randn(100)
24 ts3 = np.sin(time + np.pi/4) + 0.1 * np.random.randn(100)
25
26 # 2D data for scatter and contour
27 x = np.linspace(-3, 3, 50)
28 y = np.linspace(-3, 3, 50)
29 X, Y = np.meshgrid(x, y)
30 Z = np.sin(X) * np.cos(Y)
31
32 # Create comprehensive plot gallery
33 fig = plt.figure(figsize=(20, 15))
34 fig.suptitle('Comprehensive Plot Types Gallery', fontsize=20, font
35
36 # 1. Histogram
37 ax1 = plt.subplot(3, 4, 1)
38 ax1.hist(data_normal, bins=30, alpha=0.7, color='skyblue', edgeco
39 ax1.set_title('Histogram')
40 ax1.set_xlabel('Value')
41 ax1.set_ylabel('Frequency')
42 ax1.grid(True, alpha=0.3)
43
44 # 2. Box Plot
45 ax2 = plt.subplot(3, 4, 2)
46 box_data = [data_normal, data_exponential, data_uniform]
47 ax2.boxplot(box_data, labels=['Normal', 'Exponential', 'Uniform'])
48 ax2.set_title('Box Plot')
49 ax2.set_ylabel('Value')
50 ax2.grid(True, alpha=0.3)
51
52 # 3. Bar Chart
53 ax3 = plt.subplot(3, 4, 3)
54 ax3.bar(categories, values1, color=['red', 'green', 'blue', 'oran
55 ax3.set_title('Bar Chart')
56 ax3.set_xlabel('Categories')
57 ax3.set_ylabel('Values')
58 ax3.grid(True, alpha=0.3)
```

```

63 # 4. Scatter Plot
64 ax4 = plt.subplot(3, 4, 4)
65 x_scatter = np.random.randn(100)
66 y_scatter = 2 * x_scatter + np.random.randn(100) * 0.5
67 colors = np.random.rand(100)
68 ax4.scatter(x_scatter, y_scatter, c=colors, alpha=0.6, s=50)
69 ax4.set_title('Scatter Plot')
70 ax4.set_xlabel('X')
71 ax4.set_ylabel('Y')
72
73 # 5. Line Plot
74 ax5 = plt.subplot(3, 4, 5)
75 ax5.plot(time, ts1, 'b-', linewidth=2, label='Series 1')
76 ax5.plot(time, ts2, 'r-', linewidth=2, label='Series 2')
77 ax5.set_title('Line Plot')
78 ax5.set_xlabel('Time')
79 ax5.set_ylabel('Value')
80 ax5.legend()
81 ax5.grid(True, alpha=0.3)
82
83 # 6. Area Plot
84 ax6 = plt.subplot(3, 4, 6)
85 ax6.fill_between(time, ts1, alpha=0.7, color='blue', label='Series 1')
86 ax6.fill_between(time, ts2, alpha=0.7, color='red', label='Series 2')
87 ax6.set_title('Area Plot')
88 ax6.set_xlabel('Time')
89 ax6.set_ylabel('Value')
90 ax6.legend()
91 ax6.grid(True, alpha=0.3)
92
93 # 7. Pie Chart
94 ax7 = plt.subplot(3, 4, 7)
95 sizes = [30, 25, 20, 15, 10]
96 labels = ['A', 'B', 'C', 'D', 'E']
97 colors = ['gold', 'lightcoral', 'lightskyblue', 'lightgreen', 'plum']
98 ax7.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%', shadow=True)
99 ax7.set_title('Pie Chart')
100
101 # 8. Horizontal Bar Chart
102 ax8 = plt.subplot(3, 4, 8)
103 ax8.barh(categories, values1, color=['red', 'green', 'blue', 'orange'])
104 ax8.set_title('Horizontal Bar Chart')
105 ax8.set_xlabel('Values')
106 ax8.set_ylabel('Categories')
107 ax8.grid(True, alpha=0.3)
108
109 # 9. Stacked Bar Chart
110 ax9 = plt.subplot(3, 4, 9)
111 width = 0.35
112 x_pos = np.arange(len(categories))
113 ax9.bar(x_pos, values1, width, label='Series 1', color='skyblue')
114 ax9.bar(x_pos, values2, width, bottom=values1, label='Series 2', color='red')
115 ax9.bar(x_pos, values3, width, bottom=np.array(values1) + np.array(values2),
116         label='Series 3', color='lightgreen')
117 ax9.set_title('Stacked Bar Chart')
118 ax9.set_xlabel('Categories')
119 ax9.set_ylabel('Values')
120 ax9.set_xticks(x_pos)
121 ax9.set_xticklabels(categories)
122 ax9.legend()

```

```

127 ax9.grid(True, alpha=0.3)
128
129 # 10. Error Bar Plot
130 ax10 = plt.subplot(3, 4, 10)
131 x_error = np.arange(len(categories))
132 y_error = values1
133 yerr = [5, 3, 4, 6, 2] # Error values
134 ax10.errorbar(x_error, y_error, yerr=yerr, fmt='o', capsize=5, ca
135 ax10.set_title('Error Bar Plot')
136 ax10.set_xlabel('Categories')
137 ax10.set_ylabel('Values')
138 ax10.set_xticks(x_error)
139 ax10.set_xticklabels(categories)
140 ax10.grid(True, alpha=0.3)
141
142
143 # 11. Contour Plot
144 ax11 = plt.subplot(3, 4, 11)
145 contour = ax11.contour(X, Y, Z, levels=20)
146 ax11.clabel(contour, inline=True, fontsize=8)
147 ax11.set_title('Contour Plot')
148 ax11.set_xlabel('X')
149 ax11.set_ylabel('Y')
150
151
152 # 12. Heatmap
153 ax12 = plt.subplot(3, 4, 12)
154 heatmap_data = np.random.randn(10, 10)
155 im = ax12.imshow(heatmap_data, cmap='viridis', aspect='auto')
156 ax12.set_title('Heatmap')
157 plt.colorbar(im, ax=ax12)
158
159 plt.tight_layout()
160 plt.show()
161
162 # Advanced Plot Types
163 print("\nAdvanced Plot Types:")
164
165 # 3D Surface Plot
166 from mpl_toolkits.mplot3d import Axes3D
167
168 fig = plt.figure(figsize=(15, 5))
169
170 # 3D Surface
171 ax1 = fig.add_subplot(1, 3, 1, projection='3d')
172 ax1.plot_surface(X, Y, Z, cmap='viridis', alpha=0.8)
173 ax1.set_title('3D Surface Plot')
174 ax1.set_xlabel('X')
175 ax1.set_ylabel('Y')
176 ax1.set_zlabel('Z')
177
178
179 # 3D Scatter
180 ax2 = fig.add_subplot(1, 3, 2, projection='3d')
181 x_3d = np.random.randn(100)
182 y_3d = np.random.randn(100)
183 z_3d = np.random.randn(100)
184 colors_3d = np.random.rand(100)
185 ax2.scatter(x_3d, y_3d, z_3d, c=colors_3d, cmap='plasma')
186 ax2.set_title('3D Scatter Plot')
187 ax2.set_xlabel('X')
188 ax2.set_ylabel('Y')
189 ax2.set_zlabel('Z')
190

```

```

191
192     # 3D Line
193     ax3 = fig.add_subplot(1, 3, 3, projection='3d')
194     t = np.linspace(0, 4*np.pi, 100)
195     x_3d_line = np.sin(t)
196     y_3d_line = np.cos(t)
197     z_3d_line = t
198     ax3.plot(x_3d_line, y_3d_line, z_3d_line, 'b-', linewidth=2)
199     ax3.set_title('3D Line Plot')
200     ax3.set_xlabel('X')
201     ax3.set_ylabel('Y')
202     ax3.set_zlabel('Z')
203
204
205     plt.tight_layout()
206     plt.show()
207
208     # Financial Plots (Candlestick-style)
209     print("\nFinancial-style Plots:")
210
211     # Generate financial data
212     dates = pd.date_range('2023-01-01', periods=100, freq='D')
213     np.random.seed(42)
214     prices = 100 + np.cumsum(np.random.randn(100) * 0.5)
215     high = prices + np.random.uniform(0, 2, 100)
216     low = prices - np.random.uniform(0, 2, 100)
217     volume = np.random.uniform(1000, 5000, 100)
218
219     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8), height_ratios=[1, 1])
220
221     # Price chart
222     ax1.plot(dates, prices, 'b-', linewidth=2, label='Price')
223     ax1.fill_between(dates, low, high, alpha=0.3, color='blue', label='Volume')
224     ax1.set_title('Financial Data - Price Chart')
225     ax1.set_ylabel('Price ($)')
226     ax1.legend()
227     ax1.grid(True, alpha=0.3)
228
229     # Volume chart
230     ax2.bar(dates, volume, alpha=0.7, color='orange')
231     ax2.set_title('Volume')
232     ax2.set_xlabel('Date')
233     ax2.set_ylabel('Volume')
234     ax2.grid(True, alpha=0.3)
235
236     plt.tight_layout()
237     plt.show()
238
239     print("\n=== END OF DIFFERENT PLOT TYPES ===")

```

## 6. Introduction to Seaborn

### What is Seaborn?

Seaborn is a high-level statistical data visualization library built on top of Matplotlib. It provides a more convenient interface for creating attractive and informative statistical graphics.

## Key Advantages

- **Statistical focus:** Built-in statistical functions and distributions
- **Beautiful defaults:** Attractive color palettes and styling
- **Pandas integration:** Works seamlessly with DataFrames
- **High-level functions:** Complex plots with simple commands
- **Automatic estimation:** Built-in statistical estimation and error bars

## Seaborn vs Matplotlib

- **Matplotlib:** Low-level control, basic plots, customization
- **Seaborn:** High-level statistical plots, beautiful defaults, less customization

## Seaborn Categories

- **Distribution plots:** histplot, kdeplot, rugplot
- **Categorical plots:** boxplot, violinplot, stripplot, swarmplot
- **Relational plots:** scatterplot, lineplot, relplot
- **Matrix plots:** heatmap, clustermap
- **Regression plots:** regplot, lmpplot

In [ ...

```
1      # 6. Introduction to Seaborn - Statistical Visualization
2      print("=== INTRODUCTION TO SEABORN ===")
3
4
5      # Set Seaborn style
6      sns.set_style("whitegrid")
7      sns.set_palette("husl")
8
9      # Generate sample data
10     np.random.seed(42)
11     n_samples = 1000
12
13     # Create a comprehensive dataset
14     data = {
15         'x': np.random.randn(n_samples),
16         'y': 2 * np.random.randn(n_samples) + 1,
17         'category': np.random.choice(['A', 'B', 'C', 'D'], n_samples),
18         'value': np.random.exponential(2, n_samples),
19         'group': np.random.choice(['Group1', 'Group2', 'Group3'], n_s
20     }
21
22     df = pd.DataFrame(data)
23
24     print("Sample Dataset:")
25     print(df.head())
26     print(f"\nDataset shape: {df.shape}")
27     print(f"Columns: {df.columns.tolist()}")
28
29     # Basic Seaborn plots
30     fig, axes = plt.subplots(2, 3, figsize=(18, 12))
31     fig.suptitle('Introduction to Seaborn - Basic Plots', fontsize=16
32
33
34     # 1. Histogram with KDE
```

```

35 sns.histplot(data=df, x='x', kde=True, ax=axes[0, 0])
36 axes[0, 0].set_title('Histogram with KDE')
37
38 # 2. Scatter plot
39 sns.scatterplot(data=df, x='x', y='y', hue='category', ax=axes[0,
40 axes[0, 1].set_title('Scatter Plot with Hue')
41
42 # 3. Box plot
43 sns.boxplot(data=df, x='category', y='value', ax=axes[0, 2])
44 axes[0, 2].set_title('Box Plot')
45
46 # 4. Violin plot
47 sns.violinplot(data=df, x='category', y='value', ax=axes[1, 0])
48 axes[1, 0].set_title('Violin Plot')
49
50 # 5. Heatmap
51 correlation_data = df[['x', 'y', 'value']].corr()
52 sns.heatmap(correlation_data, annot=True, cmap='coolwarm', center
53 axes[1, 1].set_title('Correlation Heatmap')
54
55 # 6. Line plot
56 sns.lineplot(data=df.groupby('category')['value'].mean().reset_in
57 x='category', y='value', ax=axes[1, 2])
58 axes[1, 2].set_title('Line Plot (Mean Values)')
59
60 plt.tight_layout()
61 plt.show()
62
63 # Seaborn styling and themes
64 print("\nSeaborn Styling and Themes:")
65
66 # Different styles
67 styles = ['darkgrid', 'whitegrid', 'dark', 'white', 'ticks']
68 fig, axes = plt.subplots(2, 3, figsize=(18, 12))
69 fig.suptitle('Seaborn Styles', fontsize=16, fontweight='bold')
70
71 for i, style in enumerate(styles):
72     sns.set_style(style)
73     row = i // 3
74     col = i % 3
75
76     if i < len(styles):
77         sns.histplot(data=df, x='x', kde=True, ax=axes[row, col])
78         axes[row, col].set_title(f'Style: {style}')
79
80 # Remove the last empty subplot
81 if len(styles) < 6:
82     axes[1, 2].remove()
83
84 plt.tight_layout()
85 plt.show()
86
87 # Reset to default style
88 sns.set_style("whitegrid")
89
90 # Color palettes
91 print("\nSeaborn Color Palettes:")
92
93 palettes = ['deep', 'muted', 'pastel', 'bright', 'dark', 'colorbl
94 fig, axes = plt.subplots(2, 3, figsize=(18, 12))

```

```

99 fig.suptitle('Seaborn Color Palettes', fontsize=16, fontweight='b
100
101 for i, palette in enumerate(palettes):
102     row = i // 3
103     col = i % 3
104
105     sns.set_palette(palette)
106     sns.scatterplot(data=df, x='x', y='y', hue='category', ax=axe
107     axes[row, col].set_title(f'Palette: {palette}')
108
109 plt.tight_layout()
110 plt.show()
111
112 # Reset to default palette
113 sns.set_palette("husl")
114
115 # Seaborn with Pandas DataFrames
116 print("\nSeaborn with Pandas DataFrames:")
117
118 # Create a more complex dataset
119 np.random.seed(42)
120 n_samples = 500
121
122 complex_data = {
123     'age': np.random.normal(35, 10, n_samples),
124     'income': np.random.normal(50000, 15000, n_samples),
125     'education': np.random.choice(['High School', 'Bachelor', 'Ma
126     'experience': np.random.normal(10, 5, n_samples),
127     'department': np.random.choice(['IT', 'HR', 'Finance', 'Marke
128     'performance': np.random.normal(7, 2, n_samples)
129 }
130
131 df_complex = pd.DataFrame(complex_data)
132 df_complex['age'] = np.clip(df_complex['age'], 22, 65) # Clip ag
133 df_complex['experience'] = np.clip(df_complex['experience'], 0, 4
134 df_complex['performance'] = np.clip(df_complex['performance'], 1,
135
136 print("Complex Dataset Sample:")
137 print(df_complex.head())
138
139 # Create comprehensive Seaborn visualizations
140 fig, axes = plt.subplots(2, 3, figsize=(20, 12))
141 fig.suptitle('Seaborn with Complex Data', fontsize=16, fontweight
142
143 # 1. Distribution plot
144 sns.histplot(data=df_complex, x='age', hue='education', kde=True,
145 axes[0, 0].set_title('Age Distribution by Education')
146
147 # 2. Scatter plot with regression
148 sns.scatterplot(data=df_complex, x='age', y='income', hue='educat
149 sns.regplot(data=df_complex, x='age', y='income', scatter=False,
150 axes[0, 1].set_title('Age vs Income with Regression')
151
152 # 3. Box plot
153 sns.boxplot(data=df_complex, x='department', y='performance', hue
154 axes[0, 2].set_title('Performance by Department and Education')
155 axes[0, 2].tick_params(axis='x', rotation=45)
156
157 # 4. Violin plot
158 sns.violinplot(data=df_complex, x='education', y='income', ax=axe

```

```

163 axes[1, 0].set_title('Income Distribution by Education')
164 axes[1, 0].tick_params(axis='x', rotation=45)
165
166 # 5. Heatmap
167 pivot_data = df_complex.groupby(['department', 'education'])['per
168 sns.heatmap(pivot_data, annot=True, cmap='YlOrRd', ax=axes[1, 1])
axes[1, 1].set_title('Performance Heatmap')

# 6. Pair plot (simplified)
sns.scatterplot(data=df_complex, x='age', y='experience', hue='de
axes[1, 2].set_title('Age vs Experience by Department')

plt.tight_layout()
plt.show()

print("\n=== END OF SEABORN INTRODUCTION ===")

```

## 7. Statistical Plots and Distributions

### Distribution Visualization

Seaborn excels at creating statistical plots that reveal data distributions and relationships:

### Distribution Plots

- **histplot()** : Histograms with optional KDE overlay
- **kdeplot()** : Kernel Density Estimation plots
- **rugplot()** : Rug plots for showing individual data points
- **ecdfplot()** : Empirical Cumulative Distribution Function

### Statistical Estimation

- **regplot()** : Linear regression with confidence intervals
- **residplot()** : Residual plots for regression analysis
- **distplot()** : Combined histogram and KDE (deprecated, use histplot)

### Distribution Comparison

- **Multiple distributions:** Overlay different distributions
- **Faceted plots:** Separate plots for different categories
- **Statistical tests:** Built-in statistical comparisons

In [ ...

```

1 # 7. Statistical Plots and Distributions
2 print("=== STATISTICAL PLOTS AND DISTRIBUTIONS ===")
3
4
5 # Generate comprehensive statistical data
6 np.random.seed(42)
7 n_samples = 1000
8

```

```

9      # Create different distributions
10     normal_data = np.random.normal(0, 1, n_samples)
11     exponential_data = np.random.exponential(2, n_samples)
12     uniform_data = np.random.uniform(-2, 2, n_samples)
13     gamma_data = np.random.gamma(2, 1, n_samples)
14
15     # Create DataFrame for Seaborn
16     dist_data = pd.DataFrame({
17         'normal': normal_data,
18         'exponential': exponential_data,
19         'uniform': uniform_data,
20         'gamma': gamma_data
21     })
22
23
24     # Melt for easier plotting
25     dist_data_melted = dist_data.melt(var_name='distribution', value_
26
27     print("Distribution Data Sample:")
28     print(dist_data.head())
29
30     # Distribution plots
31     fig, axes = plt.subplots(2, 3, figsize=(18, 12))
32     fig.suptitle('Statistical Plots and Distributions', fontsize=16,
33
34
35     # 1. Histogram with KDE
36     sns.histplot(data=dist_data_melted, x='value', hue='distribution'
37     axes[0, 0].set_title('Histogram with KDE')
38
39     # 2. KDE Plot
40     sns.kdeplot(data=dist_data_melted, x='value', hue='distribution',
41     axes[0, 1].set_title('KDE Plot')
42
43     # 3. ECDF Plot
44     sns.ecdfplot(data=dist_data_melted, x='value', hue='distribution'
45     axes[0, 2].set_title('ECDF Plot')
46
47     # 4. Rug Plot
48     sns.rugplot(data=dist_data_melted, x='value', hue='distribution',
49     axes[1, 0].set_title('Rug Plot')
50
51     # 5. Box Plot
52     sns.boxplot(data=dist_data_melted, x='distribution', y='value', a
53     axes[1, 1].set_title('Box Plot')
54     axes[1, 1].tick_params(axis='x', rotation=45)
55
56     # 6. Violin Plot
57     sns.violinplot(data=dist_data_melted, x='distribution', y='value'
58     axes[1, 2].set_title('Violin Plot')
59     axes[1, 2].tick_params(axis='x', rotation=45)
60
61
62     plt.tight_layout()
63     plt.show()
64
65     # Regression and correlation analysis
66     print("\nRegression and Correlation Analysis:")
67
68     # Generate correlated data
69     np.random.seed(42)
70     n_samples = 200
71     x = np.random.normal(0, 1, n_samples)

```

```

73 y = 2 * x + np.random.normal(0, 0.5, n_samples)
74 z = x + y + np.random.normal(0, 0.3, n_samples)
75
76 regression_data = pd.DataFrame({
77     'x': x,
78     'y': y,
79     'z': z,
80     'category': np.random.choice(['A', 'B', 'C'], n_samples)
81 })
82
83
84 fig, axes = plt.subplots(2, 2, figsize=(15, 12))
85 fig.suptitle('Regression and Correlation Analysis', fontsize=16,
86
87 # 1. Scatter plot with regression line
88 sns.scatterplot(data=regression_data, x='x', y='y', ax=axes[0, 0])
89 sns.regplot(data=regression_data, x='x', y='y', ax=axes[0, 0])
90 axes[0, 0].set_title('Scatter Plot with Regression')
91
92 # 2. Residual plot
93 sns.residplot(data=regression_data, x='x', y='y', ax=axes[0, 1])
94 axes[0, 1].set_title('Residual Plot')
95
96 # 3. Correlation heatmap
97 correlation_matrix = regression_data[['x', 'y', 'z']].corr()
98 sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', cent
99 axes[1, 0].set_title('Correlation Heatmap')
100
101 # 4. Scatter plot with hue
102 sns.scatterplot(data=regression_data, x='x', y='y', hue='category')
103 sns.regplot(data=regression_data, x='x', y='y', scatter=False, ax
104 axes[1, 1].set_title('Scatter Plot with Categories')
105
106
107 plt.tight_layout()
108 plt.show()
109
110 # Advanced statistical plots
111 print("\nAdvanced Statistical Plots:")
112
113 # Create a more complex dataset
114 np.random.seed(42)
115 n_samples = 500
116
117 advanced_data = {
118     'value1': np.random.normal(0, 1, n_samples),
119     'value2': np.random.normal(1, 1.5, n_samples),
120     'value3': np.random.normal(2, 0.8, n_samples),
121     'category': np.random.choice(['Group A', 'Group B', 'Group C'
122     'subcategory': np.random.choice(['Type 1', 'Type 2', 'Type 3'
123 }
124
125
126 df_advanced = pd.DataFrame(advanced_data)
127
128 # Create advanced statistical visualizations
129 fig, axes = plt.subplots(2, 3, figsize=(20, 12))
130 fig.suptitle('Advanced Statistical Plots', fontsize=16, fontweigh
131
132 # 1. Distribution comparison
133 sns.histplot(data=df_advanced, x='value1', hue='category', kde=Tr
134 axes[0, 0].set_title('Distribution Comparison')
135
136

```

```

137 # 2. Multiple distributions
138 sns.kdeplot(data=df_advanced, x='value1', hue='category', ax=axes
139 sns.kdeplot(data=df_advanced, x='value2', hue='category', ax=axes
140 axes[0, 1].set_title('Multiple Distributions')
141
142 # 3. Faceted histogram
143 sns.histplot(data=df_advanced, x='value1', hue='category', kde=True
144             multiple='stack', ax=axes[0, 2])
145 axes[0, 2].set_title('Stacked Histogram')
146
147 # 4. Violin plot with split
148 sns.violinplot(data=df_advanced, x='category', y='value1', hue='s
149               split=True, ax=axes[1, 0])
150 axes[1, 0].set_title('Split Violin Plot')
151 axes[1, 0].tick_params(axis='x', rotation=45)
152
153 # 5. Box plot with swarm
154 sns.boxplot(data=df_advanced, x='category', y='value1', ax=axes[1
155 sns.swarmplot(data=df_advanced, x='category', y='value1', color='
156 axes[1, 1].set_title('Box Plot with Swarm')
157 axes[1, 1].tick_params(axis='x', rotation=45)
158
159 # 6. Strip plot
160 sns.stripplot(data=df_advanced, x='category', y='value1', hue='su
161               ax=axes[1, 2])
162 axes[1, 2].set_title('Strip Plot')
163 axes[1, 2].tick_params(axis='x', rotation=45)
164
165 plt.tight_layout()
166 plt.show()
167
168 # Statistical summary
169 print("\nStatistical Summary:")
170 print("Distribution Statistics:")
171 print(dist_data.describe())
172
173 print("\nCorrelation Matrix:")
174 print(regression_data[['x', 'y', 'z']].corr())
175
176 print("\n=== END OF STATISTICAL PLOTS AND DISTRIBUTIONS ===")

```

## 8. Categorical Plots

### Visualizing Categorical Data

Seaborn provides specialized functions for visualizing categorical data effectively:

#### Box Plots

- **boxplot()** : Statistical summaries with quartiles and outliers
- **boxenplot()** : Letter-value plots for large datasets
- **Customization:** Colors, orientations, grouping

#### Violin Plots

- **violinplot()** : Distribution shapes with box plot overlay
- **Split violins**: Compare distributions across categories
- **Inner plots**: Box plots, quartiles, or points inside violins

## Strip and Swarm Plots

- **stripplot()** : Individual data points with jitter
- **swarmplot()** : Non-overlapping points with automatic spacing
- **Combined plots**: Overlay on box plots for detailed view

## Bar Plots

- **barplot()** : Point estimates with confidence intervals
- **countplot()** : Counts of categorical variables
- **catplot()** : Faceted categorical plots

In [ ...

```

1      # 8. Categorical Plots - Visualizing Categorical Data
2      print("=== CATEGORICAL PLOTS ===")
3
4
5      # Generate comprehensive categorical data
6      np.random.seed(42)
7      n_samples = 500
8
9      # Create a complex categorical dataset
10     categorical_data = {
11         'value': np.concatenate([
12             np.random.normal(0, 1, 100),      # Group A
13             np.random.normal(1, 1.5, 100),    # Group B
14             np.random.normal(2, 0.8, 100),    # Group C
15             np.random.normal(1.5, 1.2, 100),  # Group D
16             np.random.normal(0.5, 2, 100)     # Group E
17         ]),
18         'category': ['Group A'] * 100 + ['Group B'] * 100 + ['Group C'] * 100 +
19                     ['Group D'] * 100 + ['Group E'] * 100,
20         'subcategory': np.random.choice(['Type 1', 'Type 2', 'Type 3'], n_samples),
21         'score': np.random.uniform(1, 10, n_samples),
22         'count': np.random.poisson(5, n_samples)
23     }
24
25
26     df_categorical = pd.DataFrame(categorical_data)
27
28     print("Categorical Dataset Sample:")
29     print(df_categorical.head())
30     print(f"\nDataset shape: {df_categorical.shape}")
31
32     # Basic categorical plots
33     fig, axes = plt.subplots(2, 3, figsize=(20, 12))
34     fig.suptitle('Categorical Plots - Basic Types', fontsize=16, fontweight='bold')
35
36     # 1. Box Plot
37     sns.boxplot(data=df_categorical, x='category', y='value', ax=axes[0, 0])
38     axes[0, 0].set_title('Box Plot')
39     axes[0, 0].tick_params(axis='x', rotation=45)
40
41 
```

```

42 # 2. Violin Plot
43 sns.violinplot(data=df_categorical, x='category', y='value', ax=axes[0, 1])
44 axes[0, 1].set_title('Violin Plot')
45 axes[0, 1].tick_params(axis='x', rotation=45)
46
47 # 3. Strip Plot
48 sns.stripplot(data=df_categorical, x='category', y='value', ax=axes[0, 2])
49 axes[0, 2].set_title('Strip Plot')
50 axes[0, 2].tick_params(axis='x', rotation=45)
51
52 # 4. Swarm Plot
53 sns.swarmplot(data=df_categorical, x='category', y='value', ax=axes[1, 0])
54 axes[1, 0].set_title('Swarm Plot')
55 axes[1, 0].tick_params(axis='x', rotation=45)
56
57 # 5. Bar Plot
58 sns.barplot(data=df_categorical, x='category', y='value', ax=axes[1, 1])
59 axes[1, 1].set_title('Bar Plot')
60 axes[1, 1].tick_params(axis='x', rotation=45)
61
62 # 6. Count Plot
63 sns.countplot(data=df_categorical, x='category', ax=axes[1, 2])
64 axes[1, 2].set_title('Count Plot')
65 axes[1, 2].tick_params(axis='x', rotation=45)
66
67 plt.tight_layout()
68 plt.show()
69
70 # Advanced categorical plots
71 print("\nAdvanced Categorical Plots:")
72
73 fig, axes = plt.subplots(2, 3, figsize=(20, 12))
74 fig.suptitle('Advanced Categorical Plots', fontsize=16, fontweight='bold')
75
76 # 1. Box plot with hue
77 sns.boxplot(data=df_categorical, x='category', y='value', hue='subject', ax=axes[0, 0])
78 axes[0, 0].set_title('Box Plot with Hue')
79 axes[0, 0].tick_params(axis='x', rotation=45)
80
81 # 2. Violin plot with split
82 sns.violinplot(data=df_categorical, x='category', y='value', hue='subject',
83               split=True, ax=axes[0, 1])
84 axes[0, 1].set_title('Split Violin Plot')
85 axes[0, 1].tick_params(axis='x', rotation=45)
86
87 # 3. Box plot with swarm overlay
88 sns.boxplot(data=df_categorical, x='category', y='value', ax=axes[0, 2])
89 sns.swarmplot(data=df_categorical, x='category', y='value', color='subject', ax=axes[0, 2])
90 axes[0, 2].set_title('Box Plot with Swarm Overlay')
91 axes[0, 2].tick_params(axis='x', rotation=45)
92
93 # 4. Horizontal box plot
94 sns.boxplot(data=df_categorical, y='category', x='value', ax=axes[1, 0])
95 axes[1, 0].set_title('Horizontal Box Plot')
96
97 # 5. Boxen plot (letter-value plot)
98 sns.boxenplot(data=df_categorical, x='category', y='value', ax=axes[1, 1])
99 axes[1, 1].set_title('Boxen Plot (Letter-Value)')
100 axes[1, 1].tick_params(axis='x', rotation=45)
101
102
103
104
105

```

```

106 # 6. Point plot
107 sns.pointplot(data=df_categorical, x='category', y='value', hue='
108 axes[1, 2].set_title('Point Plot')
109 axes[1, 2].tick_params(axis='x', rotation=45)
110
111 plt.tight_layout()
112 plt.show()
113
114 # Faceted categorical plots
115 print("\nFaceted Categorical Plots:")
116
117 # Create a faceted plot using catplot
118 g = sns.catplot(data=df_categorical, x='category', y='value', hue=
119                 col='subcategory', kind='box', height=4, aspect=0
120 g.fig.suptitle('Faceted Box Plots by Subcategory', y=1.02, fontsi
121 plt.show()
122
123 # Custom categorical plot
124 fig, ax = plt.subplots(figsize=(12, 8))
125
126 # Create a custom categorical visualization
127 sns.violinplot(data=df_categorical, x='category', y='value', hue=
128                split=True, inner='quart', ax=ax)
129
130 ax.set_title('Custom Violin Plot with Quartiles', fontsize=16, fo
131 ax.set_xlabel('Category', fontsize=12)
132 ax.set_ylabel('Value', fontsize=12)
133 ax.tick_params(axis='x', rotation=45)
134 ax.legend(title='Subcategory', bbox_to_anchor=(1.05, 1), loc='upp
135
136 plt.tight_layout()
137 plt.show()
138
139 # Statistical summary by category
140 print("\nStatistical Summary by Category:")
141 print(df_categorical.groupby('category')['value'].describe())
142
143 print("\n=== END OF CATEGORICAL PLOTS ===")

```

## 9. Matrix Plots and Heatmaps

### Visualizing Matrix Data

Matrix plots are essential for visualizing relationships and patterns in structured data:

#### Heatmaps

- **heatmap()** : Color-coded matrix visualization
- **Correlation matrices**: Show relationships between variables
- **Annotated heatmaps**: Add numerical values to cells
- **Custom colormaps**: Choose appropriate color schemes

#### Clustermaps

- **clustermap()** : Hierarchical clustering with heatmap

- **Row/column clustering:** Group similar rows or columns
- **Dendrograms:** Show clustering hierarchy
- **Custom clustering:** Use different distance metrics

## Matrix Plot Types

- **Correlation heatmaps:** Variable relationships
- **Confusion matrices:** Classification results
- **Distance matrices:** Similarity measures
- **Pivot table visualizations:** Aggregated data

In [ ...

```

1
2  # 9. Matrix Plots and Heatmaps - Visualizing Matrix Data
3  print("=== MATRIX PLOTS AND HEATMAPS ===")
4
5  # Generate comprehensive matrix data
6  np.random.seed(42)
7  n_samples = 200
8
9  # Create a dataset with multiple variables
10 matrix_data = {
11     'var1': np.random.normal(0, 1, n_samples),
12     'var2': np.random.normal(1, 1.5, n_samples),
13     'var3': np.random.normal(2, 0.8, n_samples),
14     'var4': np.random.normal(1.5, 1.2, n_samples),
15     'var5': np.random.normal(0.5, 2, n_samples),
16     'var6': np.random.normal(3, 1, n_samples),
17     'category': np.random.choice(['A', 'B', 'C', 'D'], n_samples),
18     'score': np.random.uniform(1, 10, n_samples)
19 }
20
21
22 df_matrix = pd.DataFrame(matrix_data)
23
24 # Calculate correlation matrix
25 correlation_matrix = df_matrix[['var1', 'var2', 'var3', 'var4', '
26
27 print("Matrix Data Sample:")
28 print(df_matrix.head())
29 print(f"\nCorrelation Matrix:")
30 print(correlation_matrix)
31
32 # Basic heatmaps
33 fig, axes = plt.subplots(2, 2, figsize=(16, 12))
34 fig.suptitle('Matrix Plots and Heatmaps', fontsize=16, fontweight
35
36
37 # 1. Basic heatmap
38 sns.heatmap(correlation_matrix, ax=axes[0, 0])
39 axes[0, 0].set_title('Basic Heatmap')
40
41 # 2. Annotated heatmap
42 sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', cent
43 axes[0, 1].set_title('Annotated Heatmap')
44
45 # 3. Custom colormap
46 sns.heatmap(correlation_matrix, annot=True, cmap='RdYlBu_r', cent
47             square=True, ax=axes[1, 0])

```

```

48 axes[1, 0].set_title('Custom Colormap (RdYlBu)')
49
50 # 4. Masked heatmap
51 mask = np.triu(np.ones_like(correlation_matrix, dtype=bool))
52 sns.heatmap(correlation_matrix, mask=mask, annot=True, cmap='viridis',
53             square=True, ax=axes[1, 1])
54 axes[1, 1].set_title('Masked Heatmap (Upper Triangle)')
55
56 plt.tight_layout()
57 plt.show()
58
59 # Advanced heatmaps
60 print("\nAdvanced Heatmaps:")
61
62 # Create a more complex dataset
63 np.random.seed(42)
64 n_samples = 300
65
66 # Generate data with some structure
67 advanced_matrix_data = {
68     'feature1': np.random.normal(0, 1, n_samples),
69     'feature2': np.random.normal(1, 1.5, n_samples),
70     'feature3': np.random.normal(2, 0.8, n_samples),
71     'feature4': np.random.normal(1.5, 1.2, n_samples),
72     'feature5': np.random.normal(0.5, 2, n_samples),
73     'feature6': np.random.normal(3, 1, n_samples),
74     'feature7': np.random.normal(1, 1.8, n_samples),
75     'feature8': np.random.normal(2.5, 0.9, n_samples),
76     'group': np.random.choice(['Group1', 'Group2', 'Group3'], n_samples),
77     'category': np.random.choice(['Type A', 'Type B', 'Type C'], n_samples),
78 }
79
80 df_advanced_matrix = pd.DataFrame(advanced_matrix_data)
81
82 # Calculate correlation matrix
83 advanced_corr = df_advanced_matrix[['feature1', 'feature2', 'feature3',
84                                     'feature4', 'feature5', 'feature6', 'feature7', 'feature8'].corr()
85
86 # Create advanced heatmap visualizations
87 fig, axes = plt.subplots(2, 2, figsize=(16, 12))
88 fig.suptitle('Advanced Heatmap Visualizations', fontsize=16, fontweight='bold')
89
90 # 1. Clustered heatmap
91 sns.heatmap(advanced_corr, annot=True, cmap='coolwarm', center=0,
92             square=True, ax=axes[0, 0])
93 axes[0, 0].set_title('Clustered Heatmap')
94
95 # 2. Diverging colormap
96 sns.heatmap(advanced_corr, annot=True, cmap='RdBu_r', center=0,
97             square=True, ax=axes[0, 1])
98 axes[0, 1].set_title('Diverging Colormap')
99
100 # 3. Sequential colormap
101 sns.heatmap(advanced_corr, annot=True, cmap='YlOrRd',
102             square=True, ax=axes[1, 0])
103 axes[1, 0].set_title('Sequential Colormap')
104
105 # 4. Custom formatting
106 sns.heatmap(advanced_corr, annot=True, cmap='viridis', center=0,
107             square=True, fmt='.2f', ax=axes[1, 1])
108
109
110
111

```

```

112 axes[1, 1].set_title('Custom Formatting')
113
114 plt.tight_layout()
115 plt.show()
116
117 # Clustermap
118 print("\nClustermap:")
119
120 # Create a clustermap
121 g = sns.clustermap(advanced_corr, annot=True, cmap='coolwarm', ce
122                    square=True, figsize=(10, 8))
123 g.fig.suptitle('Clustermap with Hierarchical Clustering', y=1.02,
124 plt.show()
125
126 # Pivot table heatmap
127 print("\nPivot Table Heatmap:")
128
129 # Create a pivot table
130 pivot_data = df_advanced_matrix.groupby(['group', 'category'])['f
131
132
133 fig, ax = plt.subplots(figsize=(10, 6))
134 sns.heatmap(pivot_data, annot=True, cmap='YlOrRd', ax=ax)
135 ax.set_title('Pivot Table Heatmap - Mean Feature1 by Group and Ca
136 plt.tight_layout()
137 plt.show()
138
139 # Distance matrix
140 print("\nDistance Matrix:")
141
142 # Calculate distance matrix
143 from scipy.spatial.distance import pdist, squareform
144 distance_matrix = squareform(pdist(advanced_corr))
145
146
147 fig, ax = plt.subplots(figsize=(8, 6))
148 sns.heatmap(distance_matrix, annot=True, cmap='viridis', ax=ax)
149 ax.set_title('Distance Matrix')
150 plt.tight_layout()
151 plt.show()
152
153 # Confusion matrix example
154 print("\nConfusion Matrix Example:")
155
156 # Generate confusion matrix data
157 np.random.seed(42)
158 confusion_data = np.random.randint(0, 4, (4, 4))
159 confusion_data = confusion_data + np.diag([10, 15, 12, 8]) # Ada
160
161
162 fig, ax = plt.subplots(figsize=(8, 6))
163 sns.heatmap(confusion_data, annot=True, cmap='Blues', ax=ax)
164 ax.set_title('Confusion Matrix')
165 ax.set_xlabel('Predicted')
166 ax.set_ylabel('Actual')
167 plt.tight_layout()
168 plt.show()
169
170 print("\n=== END OF MATRIX PLOTS AND HEATMAPS ===")

```

## 10. Subplots and Multiple Plots

# Complex Layouts

Creating sophisticated multi-panel figures requires understanding subplot management:

## Subplot Methods

- `plt.subplots()` : Create multiple subplots in a grid
- `plt.subplot()` : Add individual subplots
- `plt.subplot2grid()` : Create subplots with custom grid
- `GridSpec` : Advanced subplot layouts

## Layout Strategies

- **Grid layouts:** Regular rows and columns
- **Custom layouts:** Irregular subplot arrangements
- **Nested layouts:** Subplots within subplots
- **Shared axes:** Common x or y axes across subplots

## Multi-plot Types

- **Dashboard layouts:** Multiple related plots
- **Comparison plots:** Side-by-side comparisons
- **Detail views:** Overview with zoomed sections
- **Multi-panel figures:** Complex scientific figures

In [ ...

```
1      # 10. Subplots and Multiple Plots – Complex Layouts
2      print("=== SUBPLOTS AND MULTIPLE PLOTS ===")
3
4
5      # Generate comprehensive data for multiple plots
6      np.random.seed(42)
7      n_samples = 1000
8
9      # Create a complex dataset
10     multiplot_data = {
11         'x': np.random.normal(0, 1, n_samples),
12         'y': 2 * np.random.normal(0, 1, n_samples) + 1,
13         'z': np.random.normal(0, 1, n_samples),
14         'category': np.random.choice(['A', 'B', 'C', 'D'], n_samples),
15         'group': np.random.choice(['Group1', 'Group2', 'Group3'], n_s
16         'value': np.random.exponential(2, n_samples),
17         'time': np.linspace(0, 10, n_samples),
18         'trend': np.sin(np.linspace(0, 4*np.pi, n_samples)) + 0.1 * n
19     }
20
21
22     df_multiplot = pd.DataFrame(multiplot_data)
23
24     print("Multiplot Dataset Sample:")
25     print(df_multiplot.head())
26
27
```

```

28 # Basic subplot layouts
29 print("\nBasic Subplot Layouts:")
30
31 # 1. Simple grid layout
32 fig, axes = plt.subplots(2, 3, figsize=(18, 12))
33 fig.suptitle('Basic Subplot Layouts', fontsize=16, fontweight='bold')
34
35 # Plot different visualizations
36 axes[0, 0].hist(df_multiplot['x'], bins=30, alpha=0.7, color='skyblue')
37 axes[0, 0].set_title('Histogram')
38
39 axes[0, 1].scatter(df_multiplot['x'], df_multiplot['y'], alpha=0.5)
40 axes[0, 1].set_title('Scatter Plot')
41
42 axes[0, 2].boxplot([df_multiplot[df_multiplot['category'] == cat]
43                    for cat in df_multiplot['category'].unique()])
44 axes[0, 2].set_title('Box Plot')
45 axes[0, 2].set_xticklabels(['A', 'B', 'C', 'D'])
46
47 axes[1, 0].plot(df_multiplot['time'], df_multiplot['trend'], linecolor='red')
48 axes[1, 0].set_title('Time Series')
49
50 axes[1, 1].bar(['A', 'B', 'C', 'D'], df_multiplot['category'].value_counts())
51 axes[1, 1].set_title('Bar Chart')
52
53 axes[1, 2].pie(df_multiplot['group'].value_counts().values, labels=df_multiplot['group'].unique())
54 axes[1, 2].set_title('Pie Chart')
55
56 plt.tight_layout()
57 plt.show()
58
59 # Advanced subplot layouts
60 print("\nAdvanced Subplot Layouts:")
61
62 # 2. Custom layout with subplot2grid
63 fig = plt.figure(figsize=(20, 12))
64 fig.suptitle('Advanced Subplot Layouts', fontsize=16, fontweight='bold')
65
66 # Main plot (large)
67 ax1 = plt.subplot2grid((3, 4), (0, 0), colspan=2, rowspan=2)
68 ax1.scatter(df_multiplot['x'], df_multiplot['y'], c=df_multiplot['category'])
69 ax1.set_title('Main Scatter Plot')
70 ax1.set_xlabel('X')
71 ax1.set_ylabel('Y')
72
73 # Small plots
74 ax2 = plt.subplot2grid((3, 4), (0, 2))
75 ax2.hist(df_multiplot['x'], bins=20, alpha=0.7, color='lightblue')
76 ax2.set_title('X Distribution')
77
78 ax3 = plt.subplot2grid((3, 4), (0, 3))
79 ax3.hist(df_multiplot['y'], bins=20, alpha=0.7, color='lightcoral')
80 ax3.set_title('Y Distribution')
81
82 ax4 = plt.subplot2grid((3, 4), (1, 2))
83 ax4.boxplot([df_multiplot[df_multiplot['category'] == cat]['value']
84              for cat in df_multiplot['category'].unique()])
85 ax4.set_title('Value by Category')
86 ax4.set_xticklabels(['A', 'B', 'C', 'D'])
87
88 ax5 = plt.subplot2grid((3, 4), (1, 3))
89 ax5.pie(df_multiplot['group'].value_counts().values, labels=df_multiplot['group'].unique())
90
91

```

```

92     ax5.set_title('Group Distribution')
93
94     # Bottom plot (spans full width)
95     ax6 = plt.subplot2grid((3, 4), (2, 0), colspan=4)
96     ax6.plot(df_multiplot['time'], df_multiplot['trend'], linewidth=2)
97     ax6.fill_between(df_multiplot['time'], df_multiplot['trend'], alpha=0.5)
98     ax6.set_title('Time Series Trend')
99     ax6.set_xlabel('Time')
100    ax6.set_ylabel('Value')
101
102    plt.tight_layout()
103    plt.show()
104
105    # GridSpec for complex layouts
106    print("\nGridSpec for Complex Layouts:")
107
108    from matplotlib.gridspec import GridSpec
109
110    fig = plt.figure(figsize=(20, 12))
111    gs = GridSpec(4, 4, figure=fig)
112    fig.suptitle('GridSpec Complex Layout', fontsize=16, fontweight='bold')
113
114    # Create axes with GridSpec
115    ax1 = fig.add_subplot(gs[0, :]) # Top row, all columns
116    ax2 = fig.add_subplot(gs[1, 0]) # Second row, first column
117    ax3 = fig.add_subplot(gs[1, 1]) # Second row, second column
118    ax4 = fig.add_subplot(gs[1, 2]) # Second row, third column
119    ax5 = fig.add_subplot(gs[1, 3]) # Second row, fourth column
120    ax6 = fig.add_subplot(gs[2, :2]) # Third row, first two columns
121    ax7 = fig.add_subplot(gs[2, 2:]) # Third row, last two columns
122    ax8 = fig.add_subplot(gs[3, :]) # Bottom row, all columns
123
124    # Plot on each axis
125    ax1.plot(df_multiplot['time'], df_multiplot['trend'], linewidth=2)
126    ax1.set_title('Time Series Overview')
127
128    ax2.hist(df_multiplot['x'], bins=20, alpha=0.7, color='skyblue')
129    ax2.set_title('X Histogram')
130
131    ax3.hist(df_multiplot['y'], bins=20, alpha=0.7, color='lightcoral')
132    ax3.set_title('Y Histogram')
133
134    ax4.hist(df_multiplot['z'], bins=20, alpha=0.7, color='lightgreen')
135    ax4.set_title('Z Histogram')
136
137    ax5.hist(df_multiplot['value'], bins=20, alpha=0.7, color='gold')
138    ax5.set_title('Value Histogram')
139
140    ax6.scatter(df_multiplot['x'], df_multiplot['y'], alpha=0.6, c=df_multiplot['category'])
141    ax6.set_title('X vs Y Scatter')
142
143    ax7.scatter(df_multiplot['x'], df_multiplot['z'], alpha=0.6, c=df_multiplot['category'])
144    ax7.set_title('X vs Z Scatter')
145
146    ax8.boxplot([df_multiplot[df_multiplot['category'] == cat]['value'] for cat in categories])
147    ax8.set_title('Value by Category')
148    ax8.set_xticklabels(['A', 'B', 'C', 'D'])
149
150    plt.tight_layout()
151    plt.show()

```

```

156
157 # Shared axes
158 print("\nShared Axes:")
159
160 fig, axes = plt.subplots(2, 2, figsize=(15, 10), sharex=True, sha
161 fig.suptitle('Shared Axes Example', fontsize=16, fontweight='bold
162
163 # Plot with shared axes
164 axes[0, 0].scatter(df_multiplot['x'], df_multiplot['y'], alpha=0.
165 axes[0, 0].set_title('Plot 1')
166
167 axes[0, 1].scatter(df_multiplot['x'], df_multiplot['z'], alpha=0.
168 axes[0, 1].set_title('Plot 2')
169
170 axes[1, 0].scatter(df_multiplot['y'], df_multiplot['z'], alpha=0.
171 axes[1, 0].set_title('Plot 3')
172
173 axes[1, 1].scatter(df_multiplot['x'], df_multiplot['value'], alph
174 axes[1, 1].set_title('Plot 4')
175
176 plt.tight_layout()
177 plt.show()
178
179 # Dashboard layout
180 print("\nDashboard Layout:")
181
182 fig = plt.figure(figsize=(20, 15))
183 fig.suptitle('Data Science Dashboard', fontsize=20, fontweight='b
184
185 # Create dashboard layout
186 ax1 = plt.subplot2grid((4, 4), (0, 0), colspan=2, rowspan=2) # M
187 ax2 = plt.subplot2grid((4, 4), (0, 2), colspan=2) # Top right
188 ax3 = plt.subplot2grid((4, 4), (1, 2), colspan=2) # Middle right
189 ax4 = plt.subplot2grid((4, 4), (2, 0), colspan=2) # Bottom left
190 ax5 = plt.subplot2grid((4, 4), (2, 2), colspan=2) # Bottom right
191 ax6 = plt.subplot2grid((4, 4), (3, 0), colspan=4) # Bottom full
192
193 # Main plot - Scatter with trend
194 ax1.scatter(df_multiplot['x'], df_multiplot['y'], c=df_multiplot[
195 ax1.set_title('Main Analysis: X vs Y', fontsize=14, fontweight='b
196 ax1.set_xlabel('X Variable')
197 ax1.set_ylabel('Y Variable')
198
199 # Top right - Distribution
200 ax2.hist(df_multiplot['x'], bins=30, alpha=0.7, color='skyblue',
201 ax2.set_title('X Distribution', fontsize=12, fontweight='bold')
202 ax2.set_xlabel('X Value')
203 ax2.set_ylabel('Frequency')
204
205 # Middle right - Box plot
206 ax3.boxplot([df_multiplot[df_multiplot['category'] == cat]['value
207 ax3.set_title('Value by Category', fontsize=12, fontweight='bold'
208 ax3.set_xticklabels(['A', 'B', 'C', 'D'])
209 ax3.set_ylabel('Value')
210
211 # Bottom left - Time series
212 ax4.plot(df_multiplot['time'], df_multiplot['trend'], linewidth=2
213 ax4.fill_between(df_multiplot['time'], df_multiplot['trend'], alp
214 ax4.set_title('Time Series Trend', fontsize=12, fontweight='bold'
215 ax4.set_xlabel('Time')

```

```

220 ax4.set_ylabel('Trend Value')
221
222 # Bottom right - Correlation heatmap
223 correlation_data = df_multiplot[['x', 'y', 'z', 'value']].corr()
224 im = ax5.imshow(correlation_data, cmap='coolwarm', aspect='auto')
225 ax5.set_title('Correlation Matrix', fontsize=12, fontweight='bold')
226 ax5.set_xticks(range(len(correlation_data.columns)))
227 ax5.set_yticks(range(len(correlation_data.columns)))
228 ax5.set_xticklabels(correlation_data.columns)
229 ax5.set_yticklabels(correlation_data.columns)
230
231 # Bottom full width - Summary statistics
232 ax6.axis('off')
233 summary_text = f"""
234 Data Summary:
235 • Total samples: {len(df_multiplot):,}
  • Categories: {df_multiplot['category'].nunique()}
  • Groups: {df_multiplot['group'].nunique()}
  • X mean: {df_multiplot['x'].mean():.2f}
  • Y mean: {df_multiplot['y'].mean():.2f}
  • Value mean: {df_multiplot['value'].mean():.2f}
  • Correlation (X,Y): {df_multiplot['x'].corr(df_multiplot['y']):.
  """
236 ax6.text(0.1, 0.5, summary_text, fontsize=12, verticalalignment='
      bbox=dict(boxstyle='round', facecolor='lightgray', alpha
237
238 plt.tight_layout()
239 plt.show()
240
241 print("\n=== END OF SUBPLOTS AND MULTIPLE PLOTS ===")

```

## 11. Saving and Exporting Plots

### Production-Ready Output

Creating publication-quality plots requires proper saving and export techniques:

#### File Formats

- **PNG:** Raster format for web and presentations
- **PDF:** Vector format for publications and printing
- **SVG:** Scalable vector format for web and editing
- **EPS:** Encapsulated PostScript for publications
- **JPG:** Compressed raster format for photos

#### Resolution and Quality

- **DPI settings:** High resolution for print (300+ DPI)
- **Figure size:** Appropriate dimensions for target medium
- **Bbox settings:** Tight bounding box to remove whitespace
- **Transparency:** Alpha channel support for overlays

## Export Best Practices

- **Consistent styling:** Use style sheets for uniform appearance
- **Color management:** Consider colorblind-friendly palettes
- **File naming:** Descriptive names with version control
- **Metadata:** Include plot information in file properties

In [ ...

```
1  # 11. Saving and Exporting Plots - Production-Ready Output
2  print("=== SAVING AND EXPORTING PLOTS ===")
3
4  # Generate sample data for export examples
5  np.random.seed(42)
6  n_samples = 500
7
8
9  export_data = {
10     'x': np.random.normal(0, 1, n_samples),
11     'y': 2 * np.random.normal(0, 1, n_samples) + 1,
12     'category': np.random.choice(['A', 'B', 'C', 'D'], n_samples),
13     'value': np.random.exponential(2, n_samples),
14     'time': np.linspace(0, 10, n_samples),
15     'trend': np.sin(np.linspace(0, 4*np.pi, n_samples)) + 0.1 * n
16 }
17
18 df_export = pd.DataFrame(export_data)
19
20 print("Export Dataset Sample:")
21 print(df_export.head())
22
23 # Create a professional plot for export
24 fig, ax = plt.subplots(figsize=(12, 8))
25
26 # Main plot
27 scatter = ax.scatter(df_export['x'], df_export['y'], c=df_export['category'],
28                     cmap='viridis', alpha=0.7, s=50)
29
30 # Add trend line
31 z = np.polyfit(df_export['x'], df_export['y'], 1)
32 p = np.poly1d(z)
33 ax.plot(df_export['x'], p(df_export['x']), "r--", alpha=0.8, line
34
35 # Customize the plot
36 ax.set_title('Professional Plot for Export', fontsize=16, fontwei
37 ax.set_xlabel('X Variable', fontsize=12)
38 ax.set_ylabel('Y Variable', fontsize=12)
39 ax.legend(fontsize=12)
40 ax.grid(True, alpha=0.3)
41
42 # Add colorbar
43 cbar = plt.colorbar(scatter, ax=ax)
44 cbar.set_label('Value', fontsize=12)
45
46 # Add text annotation
47 ax.text(0.02, 0.98, f'Sample Size: {len(df_export):,}\nCorrelatio
48         transform=ax.transAxes, fontsize=10, verticalalignment='t
49         bbox=dict(boxstyle='round', facecolor='white', alpha=0.8)
```

```

54 plt.tight_layout()
55
56 # Save in different formats
57 print("\nSaving plots in different formats:")
58
59 # 1. PNG – High resolution for print
60 plt.savefig('professional_plot.png', dpi=300, bbox_inches='tight',
61             facecolor='white', edgecolor='none')
62 print("✓ Saved as PNG (300 DPI)")
63
64 # 2. PDF – Vector format for publications
65 plt.savefig('professional_plot.pdf', bbox_inches='tight',
66             facecolor='white', edgecolor='none')
67 print("✓ Saved as PDF (vector format)")
68
69 # 3. SVG – Scalable vector for web
70 plt.savefig('professional_plot.svg', bbox_inches='tight',
71             facecolor='white', edgecolor='none')
72 print("✓ Saved as SVG (scalable vector)")
73
74 # 4. EPS – Encapsulated PostScript
75 plt.savefig('professional_plot.eps', bbox_inches='tight',
76             facecolor='white', edgecolor='none')
77 print("✓ Saved as EPS (PostScript)")
78
79 # 5. JPG – Compressed format
80 plt.savefig('professional_plot.jpg', dpi=300, bbox_inches='tight',
81             facecolor='white', edgecolor='none', quality=95)
82 print("✓ Saved as JPG (compressed)")
83
84 plt.show()
85
86 # Multiple plots for export
87 print("\nCreating multiple plots for export:")
88
89 # Create a multi-panel figure
90 fig, axes = plt.subplots(2, 2, figsize=(16, 12))
91 fig.suptitle('Multi-Panel Figure for Export', fontsize=18, fontwe
92
93 # Plot 1: Scatter plot
94 axes[0, 0].scatter(df_export['x'], df_export['y'], c=df_export['v
95                     cmap='viridis', alpha=0.7)
96 axes[0, 0].set_title('Scatter Plot')
97 axes[0, 0].set_xlabel('X')
98 axes[0, 0].set_ylabel('Y')
99
100 # Plot 2: Histogram
101 axes[0, 1].hist(df_export['x'], bins=30, alpha=0.7, color='skyblu
102 axes[0, 1].set_title('X Distribution')
103 axes[0, 1].set_xlabel('X Value')
104 axes[0, 1].set_ylabel('Frequency')
105
106 # Plot 3: Box plot
107 box_data = [df_export[df_export['category'] == cat]['value'] for
108 axes[1, 0].boxplot(box_data, labels=['A', 'B', 'C', 'D'])
109 axes[1, 0].set_title('Value by Category')
110 axes[1, 0].set_xlabel('Category')
111 axes[1, 0].set_ylabel('Value')
112
113 # Plot 4: Time series

```

```

118 axes[1, 1].plot(df_export['time'], df_export['trend'], linewidth=
119 axes[1, 1].fill_between(df_export['time'], df_export['trend'], al
120 axes[1, 1].set_title('Time Series Trend')
121 axes[1, 1].set_xlabel('Time')
122 axes[1, 1].set_ylabel('Trend Value')
123
124 plt.tight_layout()
125
126 # Save multi-panel figure
127 plt.savefig('multi_panel_plot.png', dpi=300, bbox_inches='tight',
128             facecolor='white', edgecolor='none')
129 plt.savefig('multi_panel_plot.pdf', bbox_inches='tight',
130             facecolor='white', edgecolor='none')
131 print("\✓ Saved multi-panel figure as PNG and PDF")
132
133 plt.show()
134
135 # Export with custom styling
136 print("\nExport with custom styling:")
137
138 # Set custom style
139 plt.style.use('seaborn-v0_8')
140 sns.set_palette("husl")
141
142 # Create styled plot
143 fig, ax = plt.subplots(figsize=(10, 6))
144
145 # Create a more sophisticated plot
146 x = np.linspace(0, 2*np.pi, 100)
147 y1 = np.sin(x)
148 y2 = np.cos(x)
149 y3 = np.sin(x + np.pi/4)
150
151 ax.plot(x, y1, linewidth=3, label='sin(x)', alpha=0.8)
152 ax.plot(x, y2, linewidth=3, label='cos(x)', alpha=0.8)
153 ax.plot(x, y3, linewidth=3, label='sin(x +  $\pi/4$ )', alpha=0.8)
154
155 # Fill between curves
156 ax.fill_between(x, y1, alpha=0.2, color='blue')
157 ax.fill_between(x, y2, alpha=0.2, color='red')
158
159 # Customize
160 ax.set_title('Styled Plot for Export', fontsize=16, fontweight='b')
161 ax.set_xlabel('Angle (radians)', fontsize=12)
162 ax.set_ylabel('Value', fontsize=12)
163 ax.legend(fontsize=12)
164 ax.grid(True, alpha=0.3)
165
166 # Add annotations
167 ax.annotate('Peak', xy=(np.pi/2, 1), xytext=(np.pi/2 + 0.5, 1.2),
168            arrowprops=dict(arrowstyle='->', color='black', lw=2),
169            fontsize=10, ha='center')
170
171 plt.tight_layout()
172
173 # Save with custom styling
174 plt.savefig('styled_plot.png', dpi=300, bbox_inches='tight',
175             facecolor='white', edgecolor='none')
176 plt.savefig('styled_plot.pdf', bbox_inches='tight',
177             facecolor='white', edgecolor='none')
178

```

[illegible]

```

246         else:
247             fig.savefig(f'{filename_base}.{fmt}', bbox_inches='tight',
248                         facecolor='white', edgecolor='none')
249             print(f"✓ Exported {filename_base}.{fmt}")
250
251     # Create a final comprehensive plot
252     fig, axes = plt.subplots(2, 2, figsize=(16, 12))
253     fig.suptitle('Comprehensive Data Visualization', fontsize=18, fontweight='bold')
254
255     # Plot 1: Scatter with regression
256     axes[0, 0].scatter(df_export['x'], df_export['y'], c=df_export['value'],
257                       cmap='viridis', alpha=0.7)
258     z = np.polyfit(df_export['x'], df_export['y'], 1)
259     p = np.poly1d(z)
260     axes[0, 0].plot(df_export['x'], p(df_export['x']), "r--", alpha=0.5)
261     axes[0, 0].set_title('Scatter with Regression')
262     axes[0, 0].set_xlabel('X')
263     axes[0, 0].set_ylabel('Y')
264
265     # Plot 2: Distribution
266     axes[0, 1].hist(df_export['x'], bins=30, alpha=0.7, color='skyblue')
267     axes[0, 1].set_title('X Distribution')
268     axes[0, 1].set_xlabel('X Value')
269     axes[0, 1].set_ylabel('Frequency')
270
271     # Plot 3: Categorical comparison
272     box_data = [df_export[df_export['category'] == cat]['value'] for cat in categories]
273     axes[1, 0].boxplot(box_data, labels=['A', 'B', 'C', 'D'])
274     axes[1, 0].set_title('Value by Category')
275     axes[1, 0].set_xlabel('Category')
276     axes[1, 0].set_ylabel('Value')
277
278     # Plot 4: Time series
279     axes[1, 1].plot(df_export['time'], df_export['trend'], linewidth=2)
280     axes[1, 1].fill_between(df_export['time'], df_export['trend'], alpha=0.3)
281     axes[1, 1].set_title('Time Series Trend')
282     axes[1, 1].set_xlabel('Time')
283     axes[1, 1].set_ylabel('Trend Value')
284
285     plt.tight_layout()
286
287     # Export in multiple formats
288     export_plot(fig, 'comprehensive_plot', ['png', 'pdf', 'svg', 'eps'])
289
290     plt.show()
291
292     # File size comparison
293     print("\nFile size comparison:")
294     import os
295
296     files = ['professional_plot.png', 'professional_plot.pdf', 'professional_plot.svg',
297             'professional_plot.eps', 'professional_plot.jpg']
298
299     for file in files:
300         if os.path.exists(file):
301             size = os.path.getsize(file)
302             print(f"{file}: {size:,} bytes ({size/1024:.1f} KB)")
303
304     print("\n=== END OF SAVING AND EXPORTING PLOTS ===")
305     print("\n=== END OF MODULE 7: PLOTTING AND VISUALIZATION ===")

```

```
print("\nCongratulations! You have completed Module 7 on Plotting")
print("You now have comprehensive knowledge of:")
print("✓ Matplotlib fundamentals and customization")
print("✓ Seaborn statistical visualization")
print("✓ Multiple plot types and layouts")
print("✓ Professional styling and export techniques")
print("✓ Production-ready visualization skills")
```